

Montando um testbed LTE

Este tutorial apresenta o passo a passo para implantação de uma rede de comunicação móvel 4G utilizando USRP B200, srsRAN e Open5GS. Ao decorrer das próximas páginas você aprenderá a instalar os softwares necessários, dependências e modificar arquivos de configuração com os principais parâmetros da rede 4G. O documento está organizado em seções, conforme sumário apresentado a seguir. no Laboratório InComm da EMC-UFG, sendo orientado pelo Prof. Dr. Flávio Geraldo Coelho Rocha, contando com a colaboração do Doutorando Maykon Renan Pereira da Silva e efetivada pelo discente Ezequiel Reis Tavares. Agradecemos ao Prof. Dr. Kleber Vieira Cardoso por conceder o SDR e o SIM card que foram necessários para implementação deste testbed.

1. Instalando o driver da USRP
2. Instalando o srsRAN-4G da fonte
3. Construindo Open5GS a partir do código fonte
4. Obtendo MongoDB
5. Construindo Open5GS
6. Instalando a UI da Web do Open5GS
7. Instalando o gravador de simcard
8. Configurando e executando
9. Executando os binários e usando a rede configurada pelo Open5GS
10. Configurando o srsEPC para ser o core da rede
11. Executando os binários e usando a rede configurada pelo srsRAN

Hardware utilizado:

1. Servidor equipado com processador Xeon E5-1630, 16 GB de memória RAM, 1 TB de armazenamento, com sistema operacional Ubuntu 22.04.03 LTS;
2. SDR (USRP-B200);
3. SIM card (Open-cells e Sysmocom);
4. Smartphone com acesso ao 4G (Android).

Obs.: o ideal é que o item 1 seja um hardware similar ou superior e contenha uma entrada USB 3.0 ou superior.

Recomendo que ao início de cada uma das instalações utilize:

```
sudo apt-get update  
cd ~
```

Informativo: o primeiro comando remove versões antigas de pacotes instalados ou atualizações no sistema que não são mais necessárias e o segundo serve para acessar o seu diretório “home” do Linux..

1. Instalando o driver da USRP

Instale as dependências necessárias:

```
sudo apt-get install autoconf automake build-essential ccache cmake cpufrequtils doxygen  
ethtool g++ git inetutils-tools libboost-all-dev libncurses5 libncurses5-dev libusb-1.0-0  
libusb-1.0-0-dev libusb-dev python3-dev python3-mako python3-numpy python3-requests  
python3-scipy python3-setuptools python3-ruamel.yaml
```

A fonte UHD é armazenada em um repositório git. Logo, execute o seguinte comando no terminal:

```
git clone https://github.com/EttusResearch/uhd.git
```

Construir e instalar:

```
cd uhd/host  
mkdir build  
cd build  
cmake ../  
make  
sudo make install  
sudo ldconfig  
sudo uhd_usrp_probe
```

Nota 01: é normal que o terminal apresente isto mesmo com o dispositivo conectado, pois faltam as imagens.

```
root@InComm-ML110:~/uhd/host/build# sudo ldconfig  
root@InComm-ML110:~/uhd/host/build# sudo uhd_usrp_probe  
[INFO] [UHD] linux; GNU C++ version 11.4.0; Boost_107400; UHD_4.5.0.0-g471af98f  
[WARNING] [B200] EnvironmentError: IOError: Could not find path for image: usrp_b200_fw.hex  
  
Using images directory: <no images directory located>  
  
Set the environment variable 'UHD_IMAGES_DIR' appropriately or follow the below instructions to download  
the images package.  
  
Please run:  
  
"/usr/local/lib/uhd/uhd_images_downloader.py"  
Error: LookupError: KeyError: No devices found for ----->  
Empty Device Address  
root@InComm-ML110:~/uhd/host/build#
```

Para contornar tal problema execute o comando:

```
sudo /usr/local/lib/uhd/uhd_images_downloader.py  
sudo ldconfig  
sudo uhd_usrp_probe
```

Informativo: o primeiro comando faz a transferência das respectivas imagens no diretório adequado, o segundo cria links e refaz o índice das bibliotecas dinâmicas do arquivo, já o terceiro serve para inspecionar o equipamento de rádio conectado e retornar suas informações técnicas e configurações.

Tendo êxito o terminal ficará assim:

```
[INFO] [UHD] linux; GNU C++ version 11.4.0; Boost_107400; UHD_4.5.0.0-g471af98f
[INFO] [B200] Loading firmware image: /usr/local/share/uhd/images/usrp_b200_fw.hex...
[INFO] [B200] Detected Device: B200
[INFO] [B200] Loading FPGA image: /usr/local/share/uhd/images/usrp_b200_fpga.bin...
[INFO] [B200] Operating over USB 3.
[INFO] [B200] Detecting internal GPSDO....
[INFO] [GPS] No GPSDO found
[INFO] [B200] Initialize CODEC control...
[INFO] [B200] Initialize Radio control...
[INFO] [B200] Performing register loopback test...
[INFO] [B200] Register loopback test passed
[INFO] [B200] Setting master clock rate selection to 'automatic'.
[INFO] [B200] Asking for clock rate 16.000000 MHz...
[INFO] [B200] Actually got clock rate 16.000000 MHz.
```

```
uhd_config_info --version
```

Informativo: o primeiro comando é apenas para informar qual versão da UHD está instalada para caso queira fazer um controle de versão.

2. Instalando o srsRAN-4G da fonte

Instale as dependências necessárias:

```
sudo apt-get install build-essential libfftw3-dev libmbedtls-dev libboost-program-options-dev
libconfig++-dev libsctp-dev
```

A fonte srsRAN também é armazenada em um repositório git. Logo, basta fazer o mesmo que fizeste anteriormente:

```
git clone https://github.com/srsRAN/srsRAN_4G.git
```

Construir e instalar:

```
cd srsRAN_4G
mkdir build
cd build
cmake ../
make
make test
```

```
100% tests passed, 0 tests failed out of 1542
```

```
Label Time Summary:
```

```
chest    = 43.05 sec*proc (22 tests)
fec       = 0.91 sec*proc (11 tests)
ldpc      = 0.02 sec*proc (1 test)
lib       = 394.81 sec*proc (1007 tests)
lte       = 375.14 sec*proc (942 tests)
nr        = 302.51 sec*proc (359 tests)
pdcp      = 1.01 sec*proc (3 tests)
phch      = 96.56 sec*proc (485 tests)
phy       = 391.78 sec*proc (1004 tests)
polar     = 0.00 sec*proc (1 test)
rlc       = 32.05 sec*proc (13 tests)
sched     = 1.67 sec*proc (7 tests)
srsenb    = 12.07 sec*proc (10 tests)
srsue     = 17.97 sec*proc (3 tests)
turbo     = 0.89 sec*proc (9 tests)
```

```
Total Test time (real) = 754.09 sec
```

Para terminar a instalação:

```
sudo make install
```

```
srsran_install_configs.sh user
```

Informativo: o segundo comando serve para fazer a instalação dos arquivos que irão ser modificados posteriormente.

3. Construindo Open5GS a partir do código fonte

Nota 02: na seção “9” é apresentada uma segunda opção de core para utilizar.

4. Obtendo MongoDB

Informativo: o MongoDB funciona de maneira diferente dos bancos de dados relacionais tradicionais, onde os dados são armazenados em tabelas com relações predefinidas. Os dados são armazenados em documentos, que são estruturados em formato BSON e organizados em coleções. Cada documento contém todas as informações relevantes em um único lugar, o que permite uma rápida recuperação de dados e uma melhor performance em ambientes distribuídos. Ademais, ele é um requisito obrigatório para a utilização do Open5GS.

Importe a chave pública usada pelo sistema de gerenciamento de pacotes utilizando os seguintes comandos:

```
sudo apt install gnupg
```

```
curl -fsSL https://pgp.mongodb.com/server-6.0.asc | sudo gpg -o  
/usr/share/keyrings/mongodb-server-6.0.gpg --dearmor
```

Crie o arquivo de lista /etc/apt/sources.list.d/mongodb-org-6.0.list para a versão Ubuntu 22.04:

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-6.0.gpg]
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-6.0.list
```

Por fim, instale os pacotes:

```
sudo apt update
sudo apt install -y mongodb-org
```

Verifique se o software está em execução:

```
service mongod status
```

Se '/usr/bin/mongod' não estiver em execução:

```
sudo systemctl start mongod
```

Para iniciá-lo automaticamente na inicialização do sistema:

```
sudo systemctl enable mongod
```

5. Construindo Open5GS

Instale as dependências para construir o código-fonte:

```
sudo apt install python3-pip python3-setuptools python3-wheel ninja-build build-essential
flex bison libsctp-dev libgnutls28-dev libgcrypt-dev libssl-dev libidn11-dev libmongoc-dev
libbson-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev libnghttp2-dev
libtins-dev libtalloc-dev meson
```

A fonte Open5GS também é armazenada em um repositório git. Logo, basta fazer o mesmo que fizeste anteriormente:

```
git clone -b v2.6.6 https://github.com/open5gs/open5gs.git
```

Construir e instalar:

```
cd open5gs
meson build --prefix=`pwd`/install
```

Informativo: o meson é um sistema de compilação, de código aberto, que aumenta a eficiência do processo que está sendo utilizado.

```

configuring open5gs project using configuration
Message:
  prefix:                /root/open5gs/install
  libdir:                /root/open5gs/install/lib/x86_64-linux-gnu
  bindir:                /root/open5gs/install/bin
  sysconfdir:            /root/open5gs/install/etc
  localstatedir:         /root/open5gs/install/var
  build location:        /root/open5gs/build
  source location:       /root/open5gs
  compiler:              gcc
  debugging support:     debug

Build targets in project: 95

open5gs 2.6.6

Subprojects
  freeDiameter      : YES 1 warnings
  prometheus-client-c: YES

User defined options
  prefix           : /root/open5gs/install

Found ninja-1.10.1 at /usr/bin/ninja

```

ninja -C build

Verifique se a compilação está correta:

mm

```

root@InComm-ML110:~/open5gs# ./build/tests/attach/attach
ssetup-test      : SUCCESS
simple-test       : SUCCESS
guti-test        : SUCCESS
auth-test        : SUCCESS
idle-test        : SUCCESS
emm-status-test  : SUCCESS
ue-context-test  : SUCCESS
reset-test       : SUCCESS
issues-test      : SUCCESS
crash-test       : SUCCESS
All tests passed.

```

cd build

meson test -v

```

Ok:                13
Expected Fail:     0
Fail:              0
Unexpected Pass:   0
Skipped:           0
Timeout:           0

Full log written to /root/open5gs/build/meson-logs/testlog.txt

```

Obs.: se o seu terminal apresentou algum erro, refaça os passos antes de prosseguir.

cd build

ninja install

ldconfig

cd ../

6. Instalando a UI da Web do Open5GS

Baixe e importe a chave Nodesource GPG usando:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor
-o /etc/apt/keyrings/nodesource.gpg
```

Crie o repositório deb:

```
NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
https://deb.nodesource.com/node_${NODE_MAJOR}.x nodistro main" | sudo tee
/etc/apt/sources.list.d/nodesource.list
```

Execute a atualização e instale:

```
sudo apt update
sudo apt install nodejs -y
```

Por fim, instale a WebUI do Open5GS:

```
curl -fsSL https://open5gs.org/open5gs/assets/webui/install | sudo -E bash -
```

```
[Install]
WantedBy=multi-user.target
EOF
+ systemctl daemon-reload
+ systemctl enable open5gs-webui
Created symlink /etc/systemd/system/multi-user.target.wants/open5gs-webui.service → /lib/systemd/system/open5gs-webui.service.
+ deb-systemd-invoke start open5gs-webui

## Default Administrator Account [Username:admin, Password:1423]...

+ mongosh open5gs ./open5gs-2.6.6/docs/assets/webui/mongo-init.js
Current Mongosh Log ID: 652ec8f47ad3ab815b35c174
Connecting to:      mongodb://127.0.0.1:27017/open5gs?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.0.1
Using MongoDB:      6.0.10
Using Mongosh:      2.0.1

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

-----
The server generated these startup warnings when booting
2023-10-17T12:30:47.710-03:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongo
db.org/core/prodnotes-filesystem
2023-10-17T12:30:49.253-03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestrict
ed
2023-10-17T12:30:49.254-03:00: vm.max_map_count is too low
-----
Loading file: ./open5gs-2.6.6/docs/assets/webui/mongo-init.js
+ rm -rf ./open5gs-2.6.6
```

7. Instalando o gravador de simcard:

Antes de conectar o dispositivo na porta usb:

lsusb

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# lsusb
Bus 002 Device 002: ID 8087:8002 Intel Corp. 8 channel internal hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 001 Device 002: ID 8087:800a Intel Corp. Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 005 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 004 Device 008: ID 04ca:00bd Lite-On Technology Corp. Dell Wireless Device
Bus 004 Device 002: ID 0424:2660 Microchip Technology, Inc. (formerly SMSC) Hub
Bus 004 Device 006: ID 04b3:3025 IBM Corp. NetVista Full Width Keyboard
Bus 004 Device 005: ID 046d:c05a Logitech, Inc. M90/M100 Optical Mouse
Bus 004 Device 004: ID 062a:5918 MosArt Semiconductor Corp. 2.4G Keyboard Mouse
Bus 004 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Após conectar o dispositivo:

lsusb

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# lsusb
Bus 002 Device 002: ID 8087:8002 Intel Corp. 8 channel internal hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 001 Device 002: ID 8087:800a Intel Corp. Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 005 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 004 Device 009: ID 067b:2303 Prolific Technology, Inc. PL2303 Serial Port / Mobile Action MA-8910P
Bus 004 Device 008: ID 04ca:00bd Lite-On Technology Corp. Dell Wireless Device
Bus 004 Device 002: ID 0424:2660 Microchip Technology, Inc. (formerly SMSC) Hub
Bus 004 Device 006: ID 04b3:3025 IBM Corp. NetVista Full Width Keyboard
Bus 004 Device 005: ID 046d:c05a Logitech, Inc. M90/M100 Optical Mouse
Bus 004 Device 004: ID 062a:5918 MosArt Semiconductor Corp. 2.4G Keyboard Mouse
Bus 004 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Baixar software necessário para utilização do gravador em :

<https://open-cells.com/d5138782a8739209ec5760865b1e53b0/uicc-v3.2.tgz>

Colocar na pasta de preferência e descompactar, da seguinte forma:

tar -xvzf uicc-v3.2.tgz

cd uicc-v3.2

make

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# cd Downloads/
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# ls -la
.  ..  uicc-v3.2  uicc-v3.2.tgz
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# tar -xvzf uicc-v3.2.tgz
uicc-v3.2/
uicc-v3.2/README.txt
uicc-v3.2/uicc.h
uicc-v3.2/milenage.h
uicc-v3.2/program_uicc.c
uicc-v3.2/aes.h
uicc-v3.2/program_uicc_pcsc
uicc-v3.2/Makefile
uicc-v3.2/program_uicc
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# cd uicc-v3.2
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads/uicc-v3.2# make
make: 'program_uicc' está atualizado.
```


8. Configurando e executando:

➤ Gravando os dados no simcard

Depois de inserir o simcard no gravador, basta executar inserindo dados:

```
./program_uicc --adm --imsi --key --opc
```

Obs.: os valores do imsi, key e opc podem ser diversos, por isso, não foi colocado nenhum valor. Contudo, o valor de adm costuma vir descrito e necessita ser alterado com risco de bloqueio do simcard em caso de erro sucessivos do valor. por isso, não foi colocado nenhum valor.

```
root@incomm-ProLiant-ML110-Gen9:/home/incomm/Downloads/uicc-v3.2# sudo ./program_uicc --adm 0C012816 --imsi 0010100000000001 --key 82E9053A1882085FF2C020359938DAE9 --opc BFD5771AAF4F6728E9BC6EF2C2533BDB

Existing values in USIM
ICCID: 89860061100000000816
WARNING: iccid luhn encoding of last digit not done
USIM IMSI: 310789012345304
USIM MSISDN: 00000816
USIM Service Provider Name: open cells

Setting new values

Reading UICC values after uploading new values
ICCID: 89860061100000000816
WARNING: iccid luhn encoding of last digit not done
USIM IMSI: 0010100000000001
USIM MSISDN: 00000816
USIM Service Provider Name: open cells
root@incomm-ProLiant-ML110-Gen9:/home/incomm/Downloads/uicc-v3.2#
```

opc

➤ Mudando os parâmetros do Open5GS

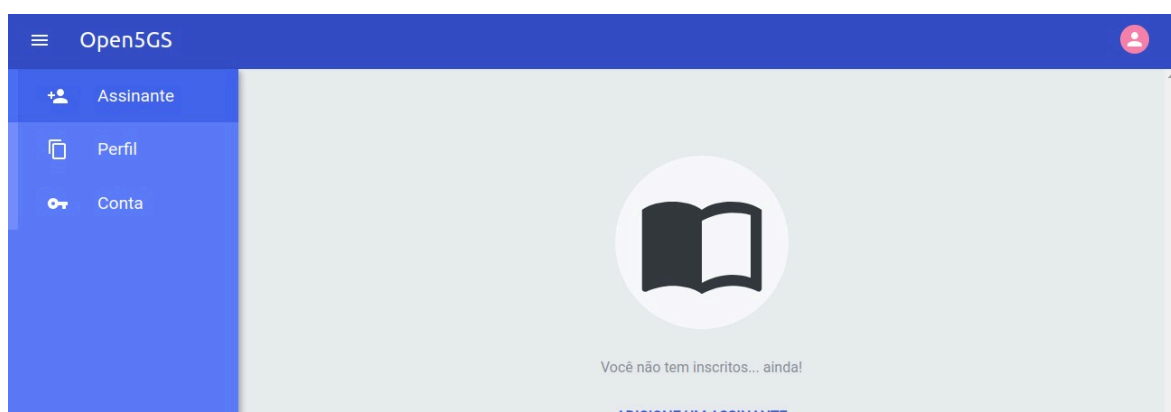
Conecte-se a <http://localhost:3000> e faça login com uma conta de administrador.

Username : admin

Password : 1423

Depois de logado:

- I. Vá para o menu do assinante;
- II. Clique no botão + para adicionar um novo assinante;
- III. Preencha o IMSI, (K, OPc, AMF) e APN do assinante e salve.



Open5GS

Editar assinante

Configuração do assinante

IMSI*

001010000000001

+

Chave de Assinante (K)*

82E9053A1882085FF2C020359938DAE9

Campo de gerenciamento de autenticação (AMF)*

8000

Tipo USIM

OPc

Chave do Operador (OPc/OP)*

BFD5771AAF4F6728E9BC6EF2C2533BDB

Link descendente UE-AMBR*

1

Unidade

Gbps

Ligação UE-AMBR*

1

Unidade

Gbps

CANCELAR

SALVAR

Modifique `install/etc/open5gs/mme.yaml` para definir o endereço IP SIAP, ID PLMN e TAC:

Padrão:

```

mme:
  freeDiameter: /root/open5gs/install/etc/freeDiameter/mme.conf
  siap:
    - addr: 127.0.0.2
  gtpc:
    - addr: 127.0.0.2
  metrics:
    - addr: 127.0.0.2
    port: 9090
  gummei:
    plmn_id:
      mcc: 999
      mnc: 70
    mme_gid: 2
    mme_code: 1
  tai:
    plmn_id:
      mcc: 999
      mnc: 70
    tac: 1
  security:
    integrity_order : [ EIA2, EIA1, EIA0 ]
    ciphering_order : [ EEA0, EEA1, EEA2 ]

```

Modificado:

```
mme:
  freeDiameter: /root/open5gs/install/etc/freeDiameter/mme.conf
  s1ap:
    - addr: 127.0.1.2
  gtpc:
    - addr: 127.0.0.2
  metrics:
    - addr: 127.0.0.2
      port: 9090
  gummei:
    plmn_id:
      mcc: 310
      mnc: 78
    mme_gid: 2
    mme_code: 1
  tai:
    plmn_id:
      mcc: 310
      mnc: 78
    tac: 5
  security:
    integrity_order : [ EIA2, EIA1, EIA0 ]
    ciphering_order : [ EEA0, EEA1, EEA2 ]
```

Obs.: foi utilizado o comando “nano” para fazer essa alteração e futuras, ainda assim, pode ser utilizado outros a depender da escolha de modificação, por exemplo, pode ser usado o vscode para não precisar ficar sempre no terminal do linux e poder voltar a versões anteriores do arquivo mais facilmente. Sugiro que use `locate` para encontrar o diretório adequado.

Após alterar os arquivos conf, reinicie os daemons Open5GS.

```
sudo systemctl restart open5gs-mmed.service
```

Para fazer a ponte entre PGWU/UPF e WAN (Internet), você deve habilitar o encaminhamento de IP e adicionar uma regra NAT às suas tabelas de IP.

Habilitar encaminhamento IPv4/IPv6:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1
```

Adicionar regra NAT:

```
sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j MASQUERADE #
sudo ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun -j
MASQUERADE
```

Configure o firewall corretamente:

```
sudo ufw status
```

Retornando “Status: active” use:

```
sudo ufw disable  
sudo ufw status
```

Retornando “Status: inactive” prossiga o tutorial..

Aviso: tu deverias considerar as configurações abaixo por motivos de segurança.

Certifique-se de que os pacotes na cadeia `INPUT` para a interface `ogstun` sejam aceitos:

```
sudo iptables -I INPUT -i ogstun -j ACCEPT
```

Impedir que os UEs se conectem ao host no qual o UPF está sendo executado:

```
sudo iptables -I INPUT -s 10.45.0.0/16 -j DROP  
sudo ip6tables -I INPUT -s 2001:db8:cafe::/48 -j DROP
```

Se a sua rede principal for executada em vários hosts, você provavelmente desejará bloquear UE originando tráfego de acesso a outras funções de rede. Substitua x.x.x.x/y pelo IP/sub-rede VNFs:

```
sudo iptables -I FORWARD -s 10.45.0.0/16 -d x.x.x.x/y -j DRO
```

➤ **Mudando os parâmetros do srsRAN**

Volte para o diretório de origem do srsRAN e modifique:

enb.conf

```
#####  
[enb]  
enb_id = 0x19B  
#mcc = 001  
#mnc = 01  
mcc = 310  
mnc = 78  
#nme_addr = 127.0.1.100  
nme_addr = 127.0.1.2  
gtp_bind_addr = 127.0.1.1  
s1c_bind_addr = 127.0.1.1  
s1c_bind_port = 0  
#n_prb = 50  
n_prb = 100  
#tm = 4  
#nof_ports = 2  
#####
```

rr.conf

```
cell_list =
(
{
// rf_port = 0;
cell_id = 0x01;
#tac = 0x0007
tac = 0x0005;
pci = 1;
// root_seq_idx = 204;
dl_earfcn = 3350;
//ul_earfcn = 21400;
ho_active = false;
//meas_gap_period = 0; // 0 (inactive), 40 or 80
//meas_gap_offset_subframe = [6, 12, 18, 24, 30];
// target_pusch_snr = -1;
// target_pucch_snr = -1;
// enable_phr_handling = false;
// min_phr_thres = 0;
// allowed_meas_bw = 6;
// t304 = 2000; // in msec. possible values: 50, 100, 150, 200, 500, 1000, 2000
```

Obs.: utilizei o comando `locate` para encontrar o diretório adequado. Além disso, os dados padrões dos arquivos foram preservados em forma de comentário conforme demonstrado nas imagens anteriores, pois serão utilizados posteriormente.

➤ Mudando os parâmetros no celular:

Retire o SIM card da sua operadora e insira o de teste que está sendo utilizado para definir os parâmetros para que fique no slot 1 do celular e crie uma APN, use os dados encontrado em “http://localhost:3000” como referência para nome e protocolo.

9. Executando os binários e usando a rede configurada pelo Open5GS

Obs.: opcionalmente, tu podes instalar e abrir o wireshark para capturas de log.

```
sudo apt-get install wireshark
sudo wireshark
```

Selecione `Loopbkacj:lo` e filtre com `slap || ngap`.

Nota 04: não é obrigatório executar cada um dos binários em terminais diferentes, porém é necessário usar pelo menos dois terminais distintos. Outrossim, com exceção de `sudo srsenb` os demais necessitam estar em um diretório específico. Portanto, use novamente o `locate` para encontrar o diretório correto e caso queira rodar apenas dois grupos de binários, basta usar `&` ao final de cada binário.

```
sudo srsenb
```

```
./open5gs-mmed
```

```
nan./open5gs-sgwcd
```

```
./open5gs-smfd
```

```
./open5gs-hssd
```

```
./open5gs-pcrfd
```

```
./open5gs-sgwud
```

```
./open5gs-upfd
```

Em caso de êxito, poderás usar a rede 4G criada:

20:34 600



Tom & Jerry em Português | Brasil | Um Pouco de Ar Fresco! | WB Kids

22 mi de visualizações há 1 ano #WBKidsPortuguês ...mais

20:36 600 78%

310 78 4G • LTE 600

CI	105217	EARFCN	68886
eNb	411	RSSI	-63 dBm
CID	1	RSRP	-86 dBm
TAC	7	RSRQ	-5 dB
PCI	1	SNR	13 dB ▲
BW	10 MHz		

600

10. Configurando o srsEPC para ser o core da rede

Durante a seção “2” acaba sendo instalado também o core do srsRAN, cujo nome é srsepc. Os passos a seguir serão para substituir o open5GS pelo srsepc no tutorial inicial.

➤ Mudando novamente os parâmetros do srsRAN

Volte para o diretório de origem do srsRAN e modifique:

Obs.: basta usar o comando `locate` e o nome do arquivo que iremos modificar.

user_db.csv

```
#
# .csv to store UE's information in HSS
# Kept in the following format: "Name,Auth,IMSI,Key,OP_Type,OP/OPc,AMF,SQN,QCI,IP_alloc"
#
# Name:      Human readable name to help distinguish UE's. Ignored by the HSS
# Auth:      Authentication algorithm used by the UE. Valid algorithms are XOR
#             (xor) and MILENAGE (mil)
# IMSI:      UE's IMSI value
# Key:       UE's key, where other keys are derived from. Stored in hexadecimal
# OP_Type:   Operator's code type, either OP or OPc
# OP/OPc:    Operator Code/Cyphered Operator Code, stored in hexadecimal
# AMF:       Authentication management field, stored in hexadecimal
# SQN:       UE's Sequence number for freshness of the authentication
# QCI:       QoS Class Identifier for the UE's default bearer.
# IP_alloc:  IP allocation strategy for the SPGW.
#             With 'dynamic' the SPGW will automatically allocate IPs
#             With a valid IPv4 (e.g. '172.16.0.2') the UE will have a statically assigned IP.
#
# Note: Lines starting by '#' are ignored and will be overwritten
ue1,mil,310789012345301,82e9053a1882085ff2c020359938dae9,opc,bfd5771aaf4f6728e9bc6ef2c2533bdb,9001,0000>
#ue1,xor,001010123456789,00112233445566778899aabbccddeeff,opc,63bfa50ee6523365ff14c1f45f88737d,9001,000>
#ue2,mil,001010123456780,00112233445566778899aabbccddeeff,opc,63bfa50ee6523365ff14c1f45f88737d,8000,000>
```

Informativo: esse arquivo em específico pode quebrar por causa do espaçamento de linhas,por isso, coloque tudo junto ou exclua os dados padrões.

ue.conf

```
#####
[usin]
mode = soft
algo = milenage
#opc = 63BFA50EE6523365FF14C1F45F88737D
#k = 00112233445566778899aabbccddeeff
#imsi = 001010123456780
#imei = 353490069873319
opc = BFD5771AAF4F6728E9BC6EF2C2533BDB
k = 82E9053A1882085FF2C020359938DAE9
imsi = 310789012345301
imei = 354251621956830
#reader =
#pin = 1234
#####
```

epc.conf

```
#####  
[mme]  
mme_code = 0x1a  
mme_group = 0x0001  
tac = 0x0007  
#mcc = 001  
#mnc = 01  
mcc = 310  
mnc = 78  
mme_bind_addr = 127.0.1.100  
apn = srsapn  
dns_addr = 8.8.8.8  
encryption_algo = EEA0  
integrity_algo = EIA1  
paging_timer = 2  
request_imei_sv = false  
lac = 0x0006  
#####
```

Obs.: lembre-se que os arquivos “**enb.conf**” e “**rr.conf**” foram modificados anteriormente, logo, basta retornar a configuração padrão da srsepc com exceção dos dados inseridos sobre o SIM card.

enb.conf

```
#####  
[enb]  
enb_id = 0x19B  
#mcc = 001  
#mnc = 01  
mcc = 310  
mnc = 78  
mme_addr = 127.0.1.100  
#mme_addr = 127.0.1.2  
gtp_bind_addr = 127.0.1.1  
s1c_bind_addr = 127.0.1.1  
s1c_bind_port = 0  
#n_prb = 50  
n_prb = 100  
#tm = 4  
#nof_ports = 2  
#####
```

rr.conf

```
cell_list =  
(  
  {  
    // rf_port = 0;  
    cell_id = 0x01;  
    #tac = 0x0005  
    tac = 0x0007;  
    pci = 1;  
    // root_seq_idx = 204;  
    dl_earfcn = 3350;  
    //ul_earfcn = 21400;  
    ho_active = false;  
    //meas_gap_period = 0; // 0 (inactive), 40 or 80  
    //meas_gap_offset_subframe = [6, 12, 18, 24, 30];  
    // target_pusch_snr = -1;  
    // target_pucch_snr = -1;  
    // enable_phr_handling = false;  
    // min_phr_thres = 0;  
    // allowed_meas_bw = 6;  
    // t304 = 2000; // in msec. possible values: 50, 100, 150, 200, 500, 1000, 2000  
    // tx_gain = 20.0; // in dB. This gain is set by scaling the source signal.
```


➤ **Criando máscara para usar a rede :**

Nota 05: sem isso não haverá uma conexão de internet funcional.

Primeiramente, descubra a interface de internet do seu dispositivo:

```
apt install net-tools  
ifconfig
```

Informativo: o primeiro comando serve para verificar todas as interfaces de rede.

Obs.: normalmente a interface a ser escolhida demonstrará tráfego de dados.

```
root@InComm-ML110:~# ifconfig  
eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 172.16.3.87 netmask 255.255.254.0 broadcast 172.16.3.255  
    inet6 fe80::4294:cb03:2e83:5e21 prefixlen 64 scopeid 0x20<link>  
    ether b0:5a:da:56:b9:18 txqueuelen 1000 (Ethernet)  
    RX packets 778112 bytes 783119596 (783.1 MB)  
    RX errors 0 dropped 22 overruns 0 frame 0  
    TX packets 488162 bytes 145388690 (145.3 MB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
    device interrupt 16  
  
eno2: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    ether b0:5a:da:56:b9:19 txqueuelen 1000 (Ethernet)  
    RX packets 1 bytes 64 (64.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2 bytes 450 (450.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
    device interrupt 17
```

Após isso, basta usar o comando `./srsepc_if_masq.sh` + a interface correspondente.

➤ **Mudando os parâmetros no celular:**

Supondo que o aparelho não sofreu alteração desde a última modificação, basta criar uma outra APN, use os dados do arquivo “epc.conf” como referência para nome e protocolo.

11. Executando os binários e usando a rede configurada pelo srsRAN

Obs.: diferente da **Nota 04** basta abrir duas janelas no mesmo terminal e executar os seguintes comandos:

```
sudo srsepc  
sudo srsenb type=x300,
```